

Q1. How can I inspect a Python function, variable by variable, in Spyder?

Question

I am new to Python. Given a function, how can I break it down in Spyder, inspect each variable's type and size, and understand what happens step by step?

The worked case is your pair of functions, `generate_data_local` and `generate_data_global`. **Pause inside the function, inspect its variables, then advance one line at a time.**

1. Understand the data model

Both functions generate `N_TRAIN` input/output pairs for $y = \exp(-x / 2) + \text{noise}$. Here `N_TRAIN = 10`, and `NOISE_STD = 4 * 0.03 = 0.12`. Inputs are drawn uniformly between 0 and 6. The Gaussian noise has mean 0 and standard deviation 0.12; its variance is 0.0144. A particular set of ten noise samples need not have sample mean 0 or sample standard deviation 0.12. See [NumPy: normal distribution parameters](#)^[1].

We expose `y_true` and `noise` as named intermediate variables so that the original expression can be inspected in three steps:

```
y_true = np.exp(-x / 2.0)
noise = rng.normal(0.0, NOISE_STD, size=(N_TRAIN, 1))
y = y_true + noise
```

This preserves the original calculation and order of random draws. Both returned arrays have shape `(10, 1)`.

2. Open and run the example

Open `01_spyder_function_inspection.py` in Spyder. Its Python environment needs NumPy; no input files are needed.

```
"""Inspect local/global random data generation in Spyder. Requires NumPy."""

import numpy as np

def generate_data_local(N_TRAIN, NOISE_STD):
    """Return x and noisy exp(-x/2), each shaped (N_TRAIN, 1).

    N_TRAIN is a positive integer; NOISE_STD is a nonnegative scalar.
    A fresh Generator seeded with 5 is created on every call.
    """
    rng = np.random.default_rng(5) # BREAKPOINT L1: before creating the local RNG
    x = rng.uniform(0.0, 6.0, size=(N_TRAIN, 1)) # BREAKPOINT L2: inspect rng
    y_true = np.exp(-x / 2.0)
    noise = rng.normal(0.0, NOISE_STD, size=(N_TRAIN, 1))
    y = y_true + noise
    return x, y # BREAKPOINT L3: inspect x, y_true, noise, and y
```

```
def generate_data_global(N_TRAIN, NOISE_STD):
    """Generate the same model using NumPy's legacy global RandomState.

    This demonstration resets and advances the global random state.
    """
    np.random.seed(5) # BREAKPOINT G1: before resetting global state
    x = np.random.uniform(0.0, 6.0, size=(N_TRAIN, 1))
    y_true = np.exp(-x / 2.0)
    noise = np.random.normal(0.0, NOISE_STD, size=(N_TRAIN, 1))
    y = y_true + noise
    return x, y # BREAKPOINT G2: inspect the global-RNG version's local variables
```

```
if __name__ == "__main__":
    # Distinct caller names keep both results available in Variable Explorer.
    x_local, y_local = generate_data_local(10, 4 * 0.03)
    print("Local Generator")
    print("x^T =", x_local.T) # (1, 10); x_local itself remains (10, 1)
    print("y^T =", y_local.T)

    x_global, y_global = generate_data_global(10, 4 * 0.03)
    print("Global RandomState")
    print("x^T =", x_global.T) # (1, 10); x_global itself remains (10, 1)
    print("y^T =", y_global.T)
    print("Same x arrays:", np.array_equal(x_local, x_global))
```

From a terminal at the repository root, you can also run:

```
python examples/01_spyder_function_inspection.py
```

The script uses `x_local`, `y_local`, `x_global`, and `y_global` in the caller so both results remain available. Reusing the caller names `x`, `y` for the second call, as in the original snippet, would replace their references to the first result.

Defining a function with `def` does not run its body; the calls in the last block do. Names assigned inside a function are normally local to that call. Importantly, **`x` and `y` are local in both functions**. The word `global` in the second function's name refers to the random-number state it uses, not the scope of `x` and `y`. See [Python: function definitions and local names](#) ^[2].

3. Pause inside the local function

1. Show **Variable Explorer** and **IPython Console** from **View > Panes**. See [Spyder: interface customization](#) ^[3].
2. Put the cursor on the executable line marked **BREAKPOINT L1** and choose **Debug > Toggle breakpoint**. Also set a breakpoint on the `return` line marked **BREAKPOINT L3**.
3. Start **Debug > Debug file**. If it pauses near the file's beginning, use **Continue** to reach L1.
4. At L1, inspect `N_TRAIN` and `NOISE_STD`. The assignment to `rng` has not executed. Use **Step over / Next** once to create it, then inspect `rng`.
5. Step through `x`, `y_true`, `noise`, and `y`. At L3, all these variables exist and `return` has not executed yet.

These instructions follow Spyder 6 documentation; your menu labels or shortcuts may differ. In the Debugger pane, select the `generate_data_local` stack frame if you previously selected another frame. Spyder can inspect variables in the paused frame. See [Spyder: Debugger](#) ^[4].

Variable Explorer displays **Name, Type, Size, Value**. Double-click x or y to inspect the ten rows. Its Size column summarizes different kinds of objects differently. If the pane filters out rng, inspect it in the console. See [Spyder: Variable Explorer](#)^[5].

Paused before this line	Newly available from the preceding assignment
rng = ...	N_TRAIN, NOISE_STD are the inputs; rng does not exist yet
x = ...	rng
y_true = ...	x
noise = ...	y_true
y = ...	noise
return x, y	y; all intermediate variables can now be inspected

4. Choose the stepping action

Action	Debugger command	Purpose
Step over / Next	next	Execute the current line without following ordinary nested calls line by line. Breakpoints can still interrupt it.
Step into	step	Enter a called Python function whose implementation you want to inspect.
Step out / Return	return	Run until the current function is about to return, then advance to its caller.
Continue	continue	Resume until another breakpoint or execution finishes.

Use the toolbar or the debugger prompt, often shown as IPdb. See [Python: debugger commands](#)^[6].

Use **Next** for the NumPy calls in this example. To practice **Step into**, pause on `x_local`, `y_local = generate_data_local(...)` and enter that call. To inspect the second function, set G1 and G2 and continue to it.

5. Inspect type and size explicitly

At L3, try these expressions in the debugging console:

Expression	Result in this example	Meaning
<code>type(N_TRAIN)</code>	<code><class 'int'></code>	Integer parameter, value 10
<code>type(NOISE_STD)</code>	<code><class 'float'></code>	Floating-point parameter, value 0.12
<code>type(rng).__name__</code>	<code>'Generator'</code>	Random-number generator object
<code>type(rng.bit_generator).__name__</code>	<code>'PCG64'</code>	Its underlying bit generator in the tested version
<code>type(x)</code>	<code><class 'numpy.ndarray'></code>	Python object type
<code>x.dtype</code>	<code>dtype('float64')</code>	Array element type

Expression	Result in this example	Meaning
<code>x.shape</code>	<code>(10, 1)</code>	10 rows, 1 column
<code>x.ndim</code>	2	Number of axes
<code>x.size</code>	10	Total number of elements
<code>len(x)</code>	10	Length of its first axis
<code>x.nbytes</code>	80	Element storage: 10 × 8 bytes
<code>x.T.shape</code>	<code>(1, 10)</code>	Shape of the transpose
<code>len(x.T)</code>	1	First-axis length of the transpose

`nbytes` excludes object overhead. `type(x)` and `x.dtype` answer different questions. See [NumPy: array attributes](#) ^[7]. `rng`, a Python integer, and a Python float do not have an array `.shape` or `.size`; inspect their type and value instead.

`print(x.T)` displays the transpose; it does not assign a new shape to `x`. `(10, 1)` is a 2D column, `(1, 10)` a 2D row, and `(10,)` a 1D array. A 1D array's `.T` remains 1D. In this example, mixing `y_true` of shape `(10, 1)` with noise of shape `(10,)` would broadcast to `(10, 10)`, rather than produce ten paired observations. See [NumPy: broadcasting](#) ^[8].

6. Give each variable a meaning

Variable	Type	Shape / size	Role
<code>N_TRAIN</code>	<code>int</code>	Scalar value 10	Number of input/output pairs
<code>NOISE_STD</code>	<code>float</code>	Scalar value 0.12	Noise standard deviation
<code>rng</code>	Generator	No array shape	Local random state; present only in <code>generate_data_local</code>
<code>x</code>	<code>ndarray, float64</code>	<code>(10, 1)</code> ; 10 elements	Input locations
<code>y_true</code>	<code>ndarray, float64</code>	<code>(10, 1)</code> ; 10 elements	Noise-free $\exp(-x / 2)$
<code>noise</code>	<code>ndarray, float64</code>	<code>(10, 1)</code> ; 10 elements	The actual noise samples used
<code>y</code>	<code>ndarray, float64</code>	<code>(10, 1)</code> ; 10 elements	Observed response, <code>y_true</code> + noise

`return x, y` returns a two-item tuple. The caller unpacks it into two names; it does not form a single `(10, 2)` array. The intermediate names `rng`, `y_true`, and `noise` do not automatically become caller variables.

For a quick numeric check, the first three entries, rounded to six decimal places, are:

Array	First three entries
x_local	4.830018, 4.847645, 3.091953
y_local	0.122099, -0.059417, 0.098112
x_global	1.331959, 5.224394, 1.240315
y_global	0.536282, 0.033789, 0.394728

Negative observations are possible because the added Gaussian noise is not restricted to positive values.

7. Compare local and global randomness

Property	generate_data_local	generate_data_global
Random-number API	Separate Generator from default_rng(5)	NumPy's shared legacy RandomState
Bit generator here	PCG64	MT19937
Effect on the legacy global random state	Does not reset or advance it	Resets it to seed 5, then advances it
Two calls with identical arguments	Repeat the same data in the tested environment	Repeat the same data in the tested environment

See [NumPy: Generator](#)^[9], [legacy random generation](#)^[10], and [np.random.seed](#)^[11].

The same seed integer does not give these two APIs the same sequence. Both functions restart their respective generators inside every call, so repeated calls are not fresh independent datasets. Here, “local” means separate state, not new randomness each time. For successive datasets, create one generator outside the function and pass it in. Record NumPy's version when exact reproduction matters; Generator streams are not guaranteed to remain identical across versions.

During debugging, inspect the stored noise variable. Calling `rng.normal(...)` again draws new noise and advances the state. Re-executing selected random-draw lines can therefore change the subsequent result.

8. Optional: inspect a scratch script line by line

```
# %% Inputs
import numpy as np
N_TRAIN = 10
NOISE_STD = 4 * 0.03

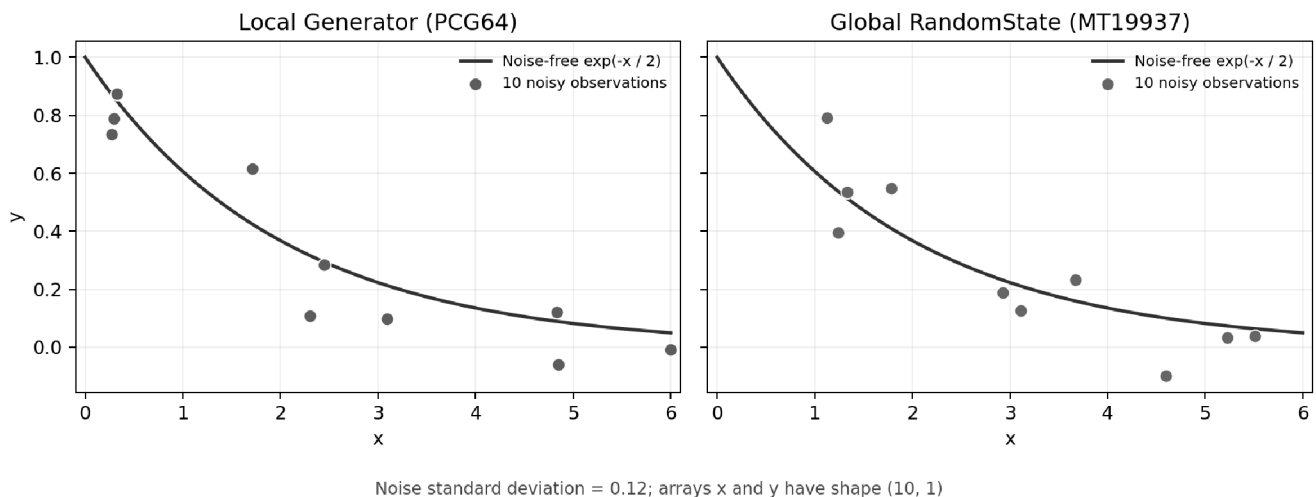
# %% Calculation
rng = np.random.default_rng(5)
x = rng.uniform(0.0, 6.0, size=(N_TRAIN, 1))
y_true = np.exp(-x / 2.0)
noise = rng.normal(0.0, NOISE_STD, size=(N_TRAIN, 1))
y = y_true + noise
```

Execute the inputs first, then use **Run selection or current line** (default F9) for each statement. **Run cell** (Windows default Ctrl+Enter) executes a # %% block. See [Spyder: running code](#)^[12]. This top-level version gives the console its own variables. Running an isolated line from a function does not automatically supply its parameters; return cannot be run at the top level.

If a variable is missing, check whether its assignment has run, whether the function already returned, and whether the correct console/frame and display filters are selected.

9. View the generated data

Same seed = 5; different random streams



Both panels use 10 observations, seed 5, and noise standard deviation 0.12. Different random streams produce different points.

The figure is generated by `02_plot_generated_data.py`. For image insertion, see [Q2: images in Markdown](#).

Verification: outputs, array shapes, equivalence to the original combined expressions, repeat calls, and global-state effects were checked using Python 3.9.25 and NumPy 1.26.4. Spyder instructions were checked against official documentation; the GUI sequence was not executed during preparation.

References

1. <https://numpy.org/doc/stable/reference/random/generated/numpy.random.Generator.normal.html>
2. <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>
3. <https://docs.spyder-ide.org/current/videos/first-steps-with-spyder.html#customization>
4. <https://docs.spyder-ide.org/current/panes/debugging.html>
5. <https://docs.spyder-ide.org/current/panes/variableexplorer.html>
6. <https://docs.python.org/3/library/pdb.html#debugger-commands>
7. <https://numpy.org/doc/stable/reference/arrays.ndarray.html#array-attributes>
8. <https://numpy.org/doc/stable/user/basics.broadcasting.html>
9. <https://numpy.org/doc/stable/reference/random/generator.html>
10. <https://numpy.org/doc/stable/reference/random/legacy.html>
11. <https://numpy.org/doc/stable/reference/random/generated/numpy.random.seed.html>
12. <https://docs.spyder-ide.org/current/panes/editor.html#running-code>